

LABORATORY ACTIVITY 8

AI Integration and Intelligent Automation for E-Commerce Systems

PageTurner Online Bookstore Management System

Subject: ITSD 82 - Web Software Tools | Section: BSIT 3C | Units: 2.0 Lecture / 4.0 Laboratory

Schedule: Thursday 1:00 PM – 3:00 PM | Lecture: CISC Room 3

1. Laboratory Background

In Laboratory Activity 7, you successfully engineered a production-scale bookstore system capable of handling 1 million+ book records with sub-second query performance, Redis caching, database partitioning, and read/write splitting. Your system now has enterprise-grade data infrastructure.

In Laboratory Activity 8, you will transform the PageTurner system into an AI-augmented intelligent platform. This lab introduces you to the industry shift toward AI-native applications—where artificial intelligence is not merely an add-on feature but a core architectural component that drives business decisions, automates operations, and enhances user experiences.

You will integrate free-tier AI services (OpenAI GPT, Google Gemini, Hugging Face Inference API, and locally-hosted Ollama models) to build intelligent automation into your bookstore system. All integrations use free-tier APIs or local models to ensure zero cost for educational purposes while demonstrating production-ready architectural patterns.

Key Difference: You are not restricted to a predefined list of modules. You will identify a real problem in your PageTurner system and design your own AI solution to solve it. The instructor's suggested modules are merely starting points—you are encouraged to innovate beyond them.

2. Learning Objectives

At the end of this laboratory activity, you will be able to:

- Identify business problems in an existing system that can be solved with AI
- Design AI-powered solutions that address real user or operational needs
- Integrate Large Language Model (LLM) APIs into a Laravel application using provider abstraction patterns
- Architect multi-provider AI systems with automatic fallback mechanisms for reliability
- Design queue-based AI pipelines for asynchronous processing of AI tasks
- Apply responsible AI practices including audit logging, bias awareness, and human oversight
- Evaluate AI service trade-offs between cloud APIs and local models for cost and performance
- Present and defend your AI integration choices with technical and business justification

3. System Scope and Requirements

3.1 Existing System Context (From Laboratory Activity 7)

Your PageTurner system currently supports:

- 1M+ book records with optimized queries (< 100ms catalog listings)
- Redis-backed caching with tag-based invalidation
- Read/write database splitting with materialized views
- Full-text search via Laravel Scout

- Chunked import/export and automated backups
- Comprehensive audit logging and API rate limiting
- Role-based dashboards (Admin, Customer, Visitor)
- Email notification workflows

3.2 Your Task: Identify and Solve a Problem with AI

Step 1: Analyze Your System

Analyze your PageTurner system and identify pain points, inefficiencies, or missed opportunities where AI could add value.

Step 2: Define Your AI Integration Concept

Determine what problem AI will solve, and for whom — customers, admins, or operations.

Step 3: Implement Your Concept

Implement your concept using free-tier AI services with production-ready architecture.

Step 4: Test, Document, and Present

Test your solution thoroughly, document your work, and present your findings.

4. Suggested AI Integration Ideas

The following are instructor suggestions to spark your creativity. You may choose one, combine multiple, or propose something entirely different. The best projects often come from your own observations of what the system needs.

SUGGESTION A AI Customer Support Chat

Problem: Customers need instant answers but human support doesn't scale.

Concept: Build a conversational AI that helps customers find books, check orders, and get recommendations through natural dialogue.

SUGGESTION B AI Executive Dashboard & Natural Language Analytics

Problem: Managers need data insights but lack SQL skills or time to analyze reports.

Concept: Let administrators ask business questions in plain English and receive automatic visualizations and insights.

SUGGESTION C	AI Sales Prediction & Inventory Optimization
Problem:	Stockouts lose revenue; overstock ties up capital.
Concept:	Predict future demand for books and automatically suggest reorder quantities and timing.
SUGGESTION D	AI Competitor & Market Intelligence
Problem:	Competitor pricing and market trends are hard to monitor manually.
Concept:	Automatically track competitor prices and generate strategic recommendations.
SUGGESTION E	AI Geographic Intelligence (GIS)
Problem:	Understanding where customers are and optimizing logistics is complex.
Concept:	Map sales geographically, optimize delivery routes, and identify expansion opportunities.
SUGGESTION F	AI Content Generation & Moderation
Problem:	Writing book descriptions, responding to reviews, and moderating content is time-consuming.
Concept:	Automatically generate marketing content and flag inappropriate user submissions for review.
SUGGESTION G	AI-Powered Personalized Recommendations
Problem:	Customers struggle to discover relevant books among 1M+ options.
Concept:	Build a recommendation engine that suggests books based on browsing history, purchase patterns, and semantic similarity.
SUGGESTION H	AI Voice Search & Accessibility
Problem:	Typing search queries is inconvenient for some users; accessibility is often overlooked.
Concept:	Enable voice-based book search and AI-generated audio descriptions for visually impaired users.
SUGGESTION I	AI Fraud Detection & Security
Problem:	Fake orders, payment fraud, and suspicious activity threaten revenue.
Concept:	Automatically flag anomalous orders and user behavior for security review.
SUGGESTION J	AI Dynamic Pricing

Problem:	Static pricing misses revenue optimization opportunities.
Concept:	Adjust book prices automatically based on demand, inventory levels, and competitor analysis.
SUGGESTION K	AI Review Summarization & Sentiment Analysis
Problem:	Thousands of reviews are unreadable at scale; sentiment is hard to track.
Concept:	Aggregate reviews into concise summaries and track sentiment trends per book/category.
SUGGESTION L	AI Automated Marketing Campaigns
Problem:	Email marketing is generic and often ignored.
Concept:	Generate personalized email content, optimize send times, and A/B test subject lines using AI.
SUGGESTION M	AI Book Discovery by Mood/Theme
Problem:	Customers know how they want to feel but not which book to read.
Concept:	"I want something uplifting about friendship" → AI suggests matching books using semantic understanding.
SUGGESTION N	AI Supply Chain & Delivery Prediction
Problem:	Delivery estimates are inaccurate; customers complain about delays.
Concept:	Predict delivery times based on weather, traffic, historical carrier performance, and geography.
SUGGESTION O	AI-Powered Learning Paths
Problem:	Customers buying technical books don't know prerequisites or reading order.
Concept:	"I want to learn Laravel" → AI generates a curated sequence of books with progression logic.

5. General Implementation Guidelines

Regardless of which AI integration concept you choose, follow these standardized phases:

PHASE 1: Discovery & Planning

Step 1: Identify Your Problem

Ask yourself: What frustrates users of my PageTurner system? What takes administrators too much time? What data do I have that could be smarter? What would make this bookstore feel "intelligent"?

Deliverable: Write a 200-word problem statement explaining: the specific problem you observed, who is affected (customers, admins, operations), why existing features don't solve it, and how AI could address it uniquely.

Step 2: Define Success Criteria

Before writing code, define what "working" means: What input does your AI feature receive? What output should it produce? How fast should it respond? What accuracy/confidence level is acceptable?

Example: "The AI chat should answer book questions correctly 80% of the time, respond within 3 seconds, and escalate to humans when uncertain."

Step 3: Select Your AI Tools

Choose based on your problem's needs:

If Your Problem Needs...	Consider Using...
Text generation, conversation, reasoning	OpenAI GPT-4o-mini, Google Gemini, Ollama
Classification (spam, sentiment, categories)	Hugging Face Inference API
Understanding images	Google Vision API, Ollama with vision models
Geographic/spatial analysis	Google Maps API
Real-time web information	Perplexity API
Unlimited free usage	Ollama (local)

Free Tier Summary

Service	Free Allowance	Best For
OpenAI GPT-4o-mini	200,000 tokens/day	Complex reasoning, chat
Google Gemini Flash	1,500 requests/day	High-volume tasks, speed
Hugging Face	Rate-limited, no hard cap	Classification, batch jobs
Google Maps	\$200 monthly credit	Geocoding, routing, maps
Perplexity	Limited free tier	Web search with citations
Ollama (self-hosted)	Unlimited	Fallback, privacy, cost control

Requirement: You must use at least one free-tier AI service and configure Ollama as a fallback for when API limits are reached.

PHASE 2: Infrastructure Setup

Step 4: Obtain API Keys

Create accounts on chosen platforms using your student email. Generate API keys from developer dashboards. Add keys to .env file—never commit to Git. Test each API with a simple request before integrating.

.env template:

```
OPENAI_API_KEY=sk-your-key
GEMINI_API_KEY=your-key
HF_API_KEY=hf-your-key
GOOGLE_MAPS_API_KEY=your-key

AI_DEFAULT_PROVIDER=openai
AI_FALLBACK_ENABLED=true

OLLAMA_ENABLED=true OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=llama3.2
```

Step 5: Install Required Packages

```
# Core AI client
composer require openai-php/client

# HTTP client (if needed)
composer require
guzzlehttp/guzzle

# For Google APIs (Gemini, Maps, Vision)
composer require google/apiclient
```

Step 6: Configure Centralized AI Management

Create config/ai.php to manage providers, rate limits, and fallbacks. Key configuration elements include: provider credentials and endpoints, rate limits for free-tier compliance, fallback chain (primary → secondary → local), feature-to-provider mapping, and cost alerting thresholds.

```
// Your AIServiceManager should support:
$ai->generate($prompt);           // Uses default provider
$ai->generate($prompt, 'chat');    // Uses provider mapped to 'chat' feature
$ai->generateWithFallback($prompt); // Tries all providers until success
```

Step 7: Set Up Ollama (Local AI Fallback)

Ollama provides unlimited free AI on your local machine.

Installation:

- Download from ollama.com
- Install and verify: `ollama --version`
- Pull models: `ollama pull llama3.2` (General chat), `ollama pull nomic-embed-text` (semantic search), `ollama pull llava` (image analysis)
- Test: `ollama run llama3.2` — Keep Ollama running during development and demos

Why this matters: When you exhaust free API limits (common during testing), Ollama ensures your system still functions. This is **required for grading reliability**.

PHASE 3: Core Implementation

Step 8: Design Your AI Service Layer

Regardless of your concept, structure your code following this pattern:

```
app/
■■■ Services/
```

```

■ ■■■ AIServiceManager.php      # Provider abstraction & fallback
■ ■■■ YourAIConceptService.php  # YOUR implementation
■ ■■■ ... other services

```

Your service class should: accept user input in a structured way, retrieve relevant context from your database (if needed), build an effective prompt for the AI, call AI through the manager (with fallback), parse and validate the AI response, transform output for your application, and log usage and decisions for audit.

Step 9: Implement Your Specific Feature

For chat/conversational features:

- Store conversation history (Redis or database)
- Implement streaming responses (SSE) for real-time feel
- Classify user intent to route to correct handlers
- Retrieve relevant data (RAG) before generating responses

For analytics/prediction features:

- Gather historical data from your seeded 1M records
- Use statistical baselines enhanced with AI insights
- Present results with confidence scores
- Schedule automated generation using Laravel Scheduler

For content generation features:

- Queue generated content for human review
- Implement approval workflow in admin dashboard
- A/B test AI vs. human content performance
- Track generation quality over time

Step 10: Build the User Interface

Your AI feature needs an interface appropriate to its users:

User Type	Interface Examples
Customers	Chat widget, voice input, recommendation cards, search results
Administrators	Dashboard panels, review queues, configuration screens
Operations	Report viewers, alert systems, batch operation status

Frontend requirements: loading states while AI processes, error messages when AI is unavailable, clear indication when content is AI-generated, mobile-responsive design, and accessibility compliance (ARIA labels, keyboard navigation).

PHASE 4: Resilience & Operations

Step 11: Implement Multi-Provider Fallback

Your system must not fail when one AI service is down or rate-limited.

Required behavior:

- Try primary provider (e.g., OpenAI)
- If rate limit exceeded or error, try secondary (e.g., Gemini)
- If all cloud providers fail, use Ollama locally
- If all AI fails, return graceful error message to user

```

public function generateWithFallback(string $prompt): string {
    $providers = config('ai.fallback_chain'); // ['openai', 'gemini', 'ollama']
    foreach ($providers as $provider) {
        try {
            if ($this->isAvailable($provider)) {
                return $this->callProvider($provider, $prompt);
            }
        } catch (\Exception $e) {
            Log::warning("{ $provider } failed: " . $e->getMessage());
            continue; // Try next provider
        }
    }
    throw new \RuntimeException('All AI providers unavailable');
}

```

Step 12: Add Queue-Based Processing

Move heavy AI tasks to background processing. When to use queues: batch processing (moderating 100 reviews, generating 50 descriptions), scheduled reports (daily insights, weekly competitor analysis), non-urgent tasks (embedding generation, retraining), and any AI call that might take > 5 seconds.

```

# Create job
php artisan make:job ProcessAITask

# Dispatch with delay ProcessAITask::dispatch($data)->delay(now()->addMinutes(5));

# Run workers
php artisan queue:work --queue=ai-tasks

```

Step 13: Implement Cost Tracking

Monitor your free-tier usage to avoid service interruption. Log every AI call and alert when approaching limits:

```

AIUsageLog::create([
    'provider' => $provider,
    'feature' => $featureName,
    'tokens_used' => $tokens, 'cost_estimate' => $this->calculateCost($provider, $tokens),
    'created_at' => now(),
]);

```

Step 14: Set Up Audit Logging All

AI decisions must be traceable:

```

Log::channel('ai_audit')->info('AI Decision', [
    'feature' => 'content_generation',
    'input_hash' => md5($input),
    'output_hash' => md5($output),
    'confidence' => $confidence,
    'provider_used' => $provider,
    'user_id' => auth()->id(),
    'timestamp' => now(),
]);

```

PHASE 5: Testing & Validation

Step 15: Test Your Implementation

Functional Testing:

- Feature works with expected inputs
- Feature handles edge cases gracefully
- Fallback activates when primary provider fails
- Queue jobs complete without errors
- Human review workflow functions (if applicable)

Performance Testing:

- AI response time is acceptable (< 5 seconds for user-facing features)
- System remains responsive during AI processing
- Queue doesn't back up under normal load
- Database queries remain optimized (< 100ms from Lab 7)

Security Testing:

- API keys are not exposed in code or logs
- User inputs are sanitized before AI processing
- AI outputs are escaped before rendering
- SQL injection is prevented (if generating queries)
- Rate limiting prevents abuse

Resilience Testing:

- Feature works with Ollama only (simulate API outage)
- Feature degrades gracefully when AI is unavailable
- Data integrity is maintained during failures
- Queue jobs retry correctly and don't lose data

6. Student-Proposed AI Integration Process

If none of the suggestions fit your vision, propose your own:

Step 1: Submit Concept Proposal

Write a 300-word proposal including:

- **Problem Statement:** What specific issue will your AI solve?
- **Target Users:** Who benefits (customers, admins, both)?
- **AI Approach:** Which AI technologies will you use and why?
- **Expected Outcome:** What does success look like?
- **Feasibility:** Can this be built with free-tier tools in the lab timeframe?

Step 2: Instructor Review

The instructor will review your proposal within 24 hours and provide feasibility feedback, technical suggestions, resource recommendations, and approval to proceed or request for refinement.

Step 3: Implementation

Follow the same Phase 1–5 guidelines above, adapted to your concept.

7. Database Schema Guidelines

Create migrations appropriate to your concept. Common needs include:

Table Purpose	When Needed
ai_conversations / ai_messages	Chat features
ai_insights / ai_reports	Analytics, predictions
ai_usage_logs	All projects (cost tracking)
ai_content_queue	Content generation with human review
competitor_intelligence	Market analysis features
geographic_sales_data	GIS/spatial features
embeddings / vector_cache	Semantic search

Requirements for all tables:

- Timestamps (created_at, updated_at)
- Foreign keys with constraints
- JSON columns for flexible AI output
- Indexes on query patterns
- Soft deletes where appropriate

8. Deliverables

8.1 Working Implementation

- Functional AI feature integrated into PageTurner
- Multi-provider fallback configured
- Queue processing implemented (if applicable)
- Database migrations and seeders
- API routes and controllers
- User interface (frontend)

8.2 Technical Documentation (1500+ words)

- Problem Identification: What problem did you choose and why?
- Solution Design: How does AI solve this problem uniquely?
- Architecture Decisions: Why did you choose specific providers, patterns, and data flows?
- Implementation Details: Key technical challenges and solutions
- Testing Results: Performance benchmarks, accuracy metrics, resilience testing
- Cost Analysis: Free-tier usage, fallback activation frequency
- Responsible AI: What safeguards did you implement?
- Future Improvements: What would you add with more time/resources?

8.3 Presentation (5 minutes)

- Live demonstration of your AI feature
- Explanation of the problem and your solution
- Technical architecture overview
- Discussion of challenges and learnings
- Q&A; with instructor and peers

8.4 Screenshots/Recordings

- Feature in action (user interactions)
- Admin interfaces (if applicable)
- Fallback mechanism working
- Queue processing status
- Cost tracking dashboard

9. Grading Rubric

Component	Weight	Criteria
Problem Identification & Creativity	15%	Clear problem statement, appropriate AI application, originality of
Technical Implementation	30%	Functional feature, proper AI integration, clean architecture, worki
Code Quality & Best Practices	15%	Laravel conventions, separation of concerns, security, documenta
Resilience & Operations	15%	Multi-provider fallback, queue usage, cost tracking, audit logging
Testing & Validation	10%	Functional tests, performance benchmarks, error handling verification
Documentation & Presentation	15%	Technical report quality, clear presentation, Q&A handling

Resources

Official Documentation

- OpenAI Platform
- Google AI Studio
- Hugging Face Inference API
- Laravel Queues Documentation
- Ollama GitHub Repository

Community & Learning

- Laravel News (AI integration articles)
- r/MachineLearning and r/Laravel subreddits
- OpenAI Community Forum
- Laravel Discord server

Final Notes

The goal of this laboratory is not to build a perfect AI system. It is to:

- Experience the process of integrating AI into a real application
- Understand the trade-offs between cloud APIs and local models
- Appreciate the importance of fallback architecture and cost management
- Practice responsible AI implementation with proper safeguards
- Develop the ability to identify AI-appropriate problems in any system

Your creativity matters. The most interesting projects often come from observing a small frustration in the existing system and asking: "Could AI make this better?" Start with the problem, not the technology. Don't choose OpenAI first and then look for something to do with it. Find a genuine pain point in your PageTurner system, then select the AI tool that solves it best.

Good luck. Build something intelligent, responsible, and genuinely useful.